Programming for Finance

Descriptive Statistics for Financial Analysis

Department of Economics

Ivan Bondarev

MGIMO UNIVERSITY

Email

- i.bondarev@inno.mgimo.ru

Signs

– Please take pictures of the information. Medium priority.

Source: https://www.flaticon.com/ru/free–icon/camera–flash_3460307

– Please write down the information. High priority.

Source: https://openmoji.org/library/emoji–270D/

TOPICS

1. Jupyter Notebook
2. CSV file structure
3. Loading and inspecting datasets
4. Descriptive statistics

Before you build complex models, you must understand your data.

Descriptive statistics is the foundation of all quantitative finance.

Jupyter Notebook

- Interactive Python Environment
- Combine code, text, and visualizations in one document
- Run code cell-by-cell (great for experimentation)
- Display outputs inline (tables, charts, statistics)
- Easy to share and document your analysis

Jupyter Notebook - Basic Commands

- Shift+Enter → Run cell and move to next
- Ctrl+Enter → Run cell and stay
- Esc + M → Convert to Markdown cell
- Esc + Y → Convert to Code cell
- B → Add new cell
- Double type D → Delete selected cells

CSV File Structure

- CSV = Comma-Separated Values

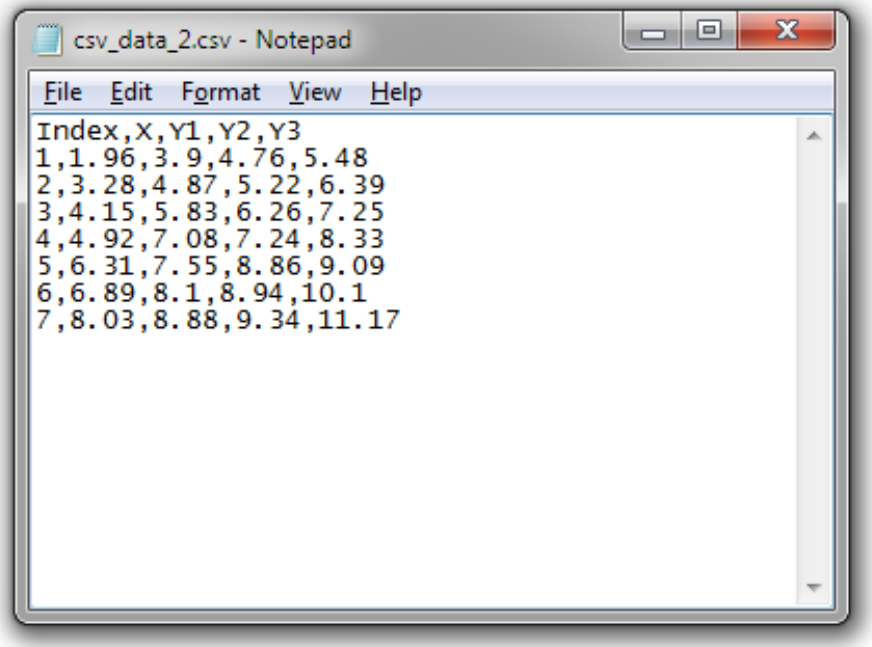
- What It Looks Like:

date,open,high,low,close,volume

2023-01-02,130.28,130.90,124.17,125.07,112117500

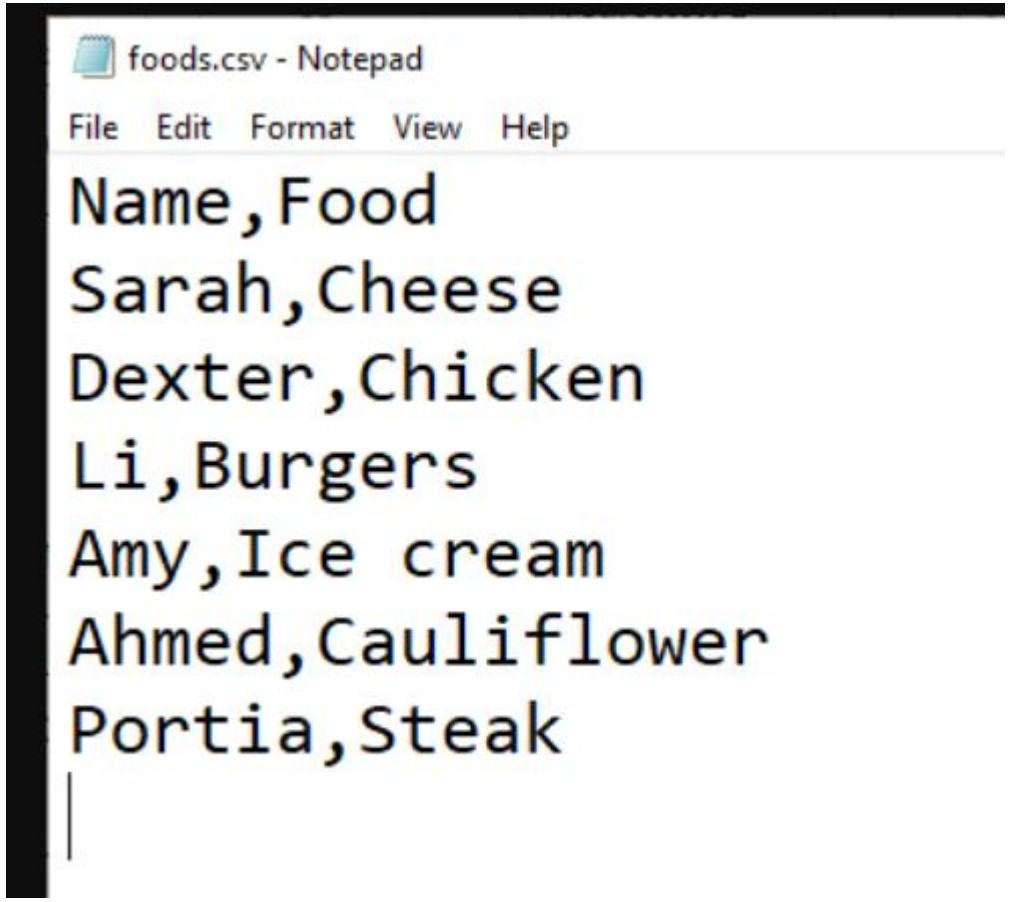
2023-01-03,126.89,128.66,125.08,125.02,89113600

2023-01-04,127.13,127.77,124.76,126.36,80962700



CSV File Structure

- First row = Column headers (variable names)
- Each subsequent row = One observation (e.g., one trading day)
- Values separated by commas (or sometimes tabs, semicolons)
- No formatting (plain text - works everywhere)



Loading and inspecting datasets

- **Essential Libraries:**

```
import pandas as pd      # Data manipulation
import numpy as np       # Numerical operations
```

- **Load CSV File:**

```
# Basic loading
df = pd.read_csv('filename.csv')


# With options
df = pd.read_csv('filename.csv',
                 index_col='date',      # Use date as index
                 parse_dates=['date'])  # Parse dates properly
```

Verify Load Success

```
print(df.head())      # First 5 rows
print(df.shape)       # (rows, columns)
print(df.columns)     # Column names
```

```
# Display first few rows to inspect structure
df.head(10)
```

Out[53]:

	carat	colour	clarity	certification	price
0	0.30	D	VS2	GIA	1302
1	0.30	E	VS1	GIA	1510
2	0.30	G	VVS1	GIA	1510
3	0.30	G	VS1	GIA	1260
4	0.31	D	VS1	GIA	1641
5	0.31	E	VS1	GIA	1555
6	0.31	F	VS1	GIA	1427
7	0.31	G	VVS2	GIA	1427
8	0.31	H	VS2	GIA	1126
9	0.31	I	VS1	GIA	1126

Initial Data Inspection

```
# 1. Check dimensions
print(f"Shape: {df.shape}")
# (308 rows, 5 columns)

# 2. View column names
print(df.columns.tolist())

# 3. Check data types
print(df.dtypes)

# 4. Check for missing values
print(df.isnull().sum())

# 5. Preview data
print(df.head(10))
```

Is data complete? Expected # of rows?

Does data look reasonable?

Descriptive Statistics - Numerical Variables

Statistic	Formula	Financial Meaning
Mean	df.mean()	Average return/price
Median	df.median()	Typical value (robust to outliers)
Std Dev	df.std()	Volatility/Risk
Variance	df.var()	Squared volatility
Min/Max	df.min()/max()	Price range, extremes
Quartiles	df.quantile()	Distribution shape
Skewness	df.skew()	Asymmetry (tail risk)
Kurtosis	df.kurtosis()	Fat tails (crash risk)

```
df.describe()  # Quick overview of all numerical columns
```

Mean vs Median

- mean_price = df['price'].mean()
- mean_return = df['return'].mean()

- Sensitive to outliers
- A few extreme values can distort the mean
- Example: One day with +50% return skews average

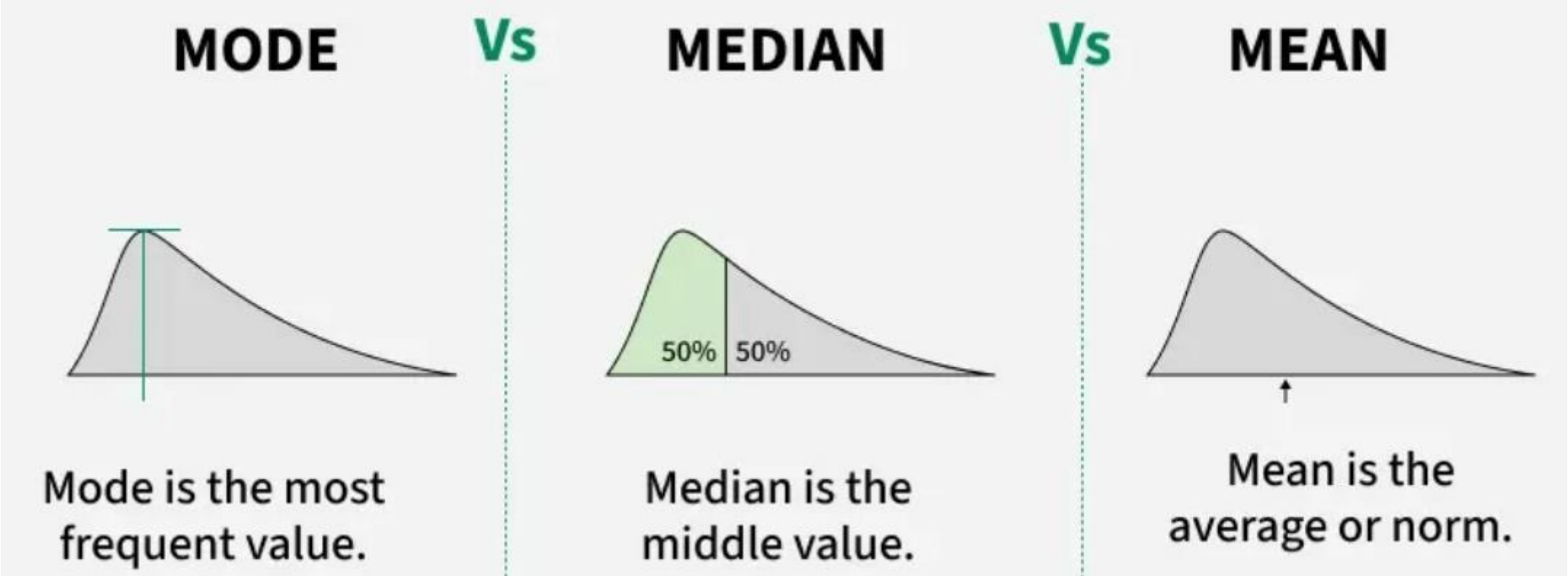
Mean vs Median

- median_price = df['price'].median()
- median_return = df['return'].median()

- Robust to outliers
- Better for skewed distributions (like prices)
- Example: Typical daily return is better represented by median

Mean vs Median

- If mean >> median → Right-skewed (common for prices)
- If mean << median → Left-skewed
- If mean ≈ median → Symmetric (like returns often are)



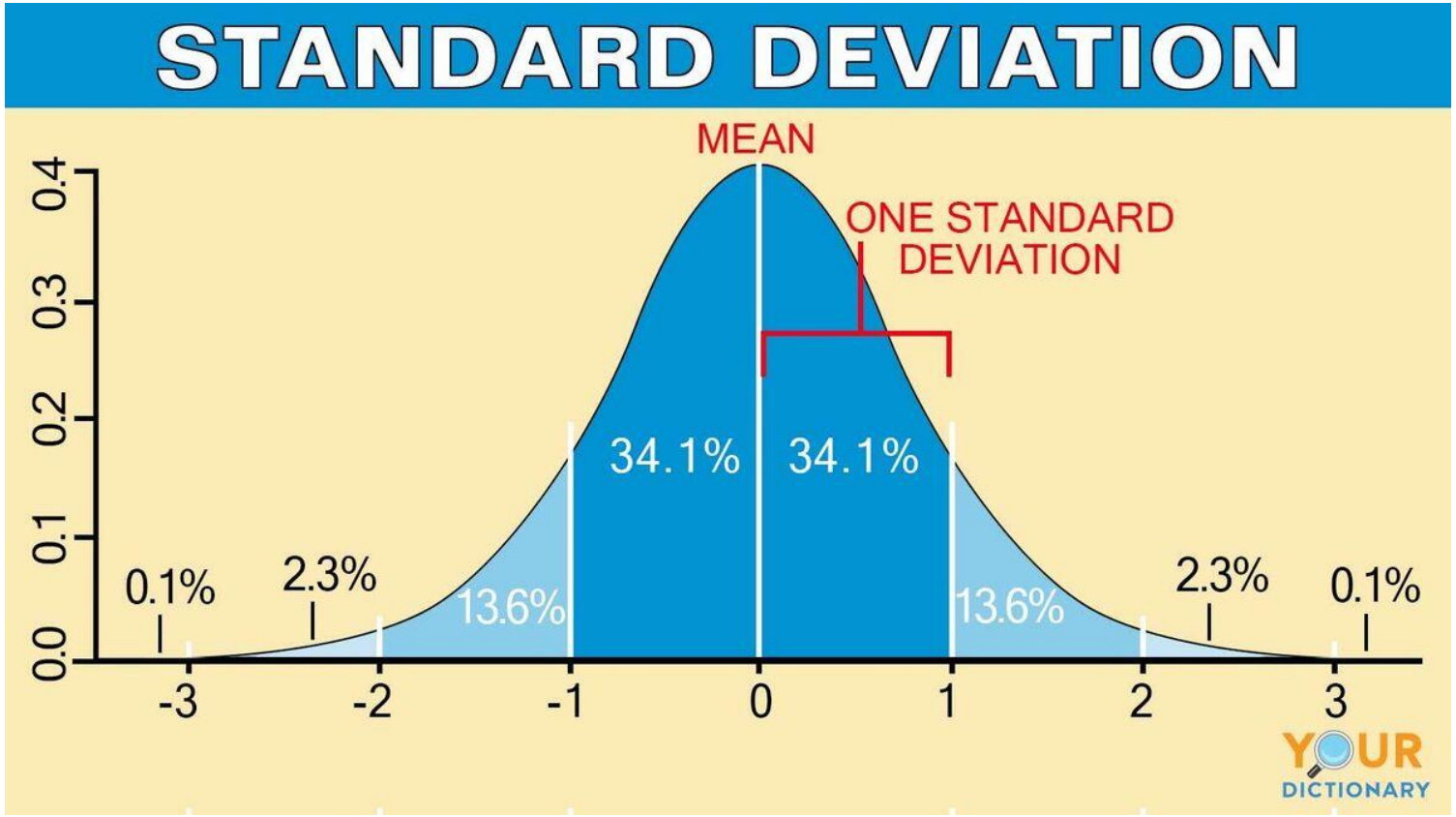
Volatility - Standard Deviation

- **# Daily volatility**
  - daily_vol = df['return'].std()
- **# Annualized volatility (for stocks)**
  - annual_vol = df['return'].std() * np.sqrt(252)
- **The Most Important Risk Metric!**

Volatility - Standard Deviation

- Why Annualize?
  - 252 = Trading days per year
  - Allows comparison across assets
  - Industry standard for reporting

- Example:
  - Stock A: 20% annual volatility
  - Stock B: 40% annual volatility
  - → Stock B is twice as risky!



Understanding Distribution Shape

- **# Quartiles (25%, 50%, 75%)**
  - quartiles = df['price'].quantile([0.25, 0.50, 0.75])

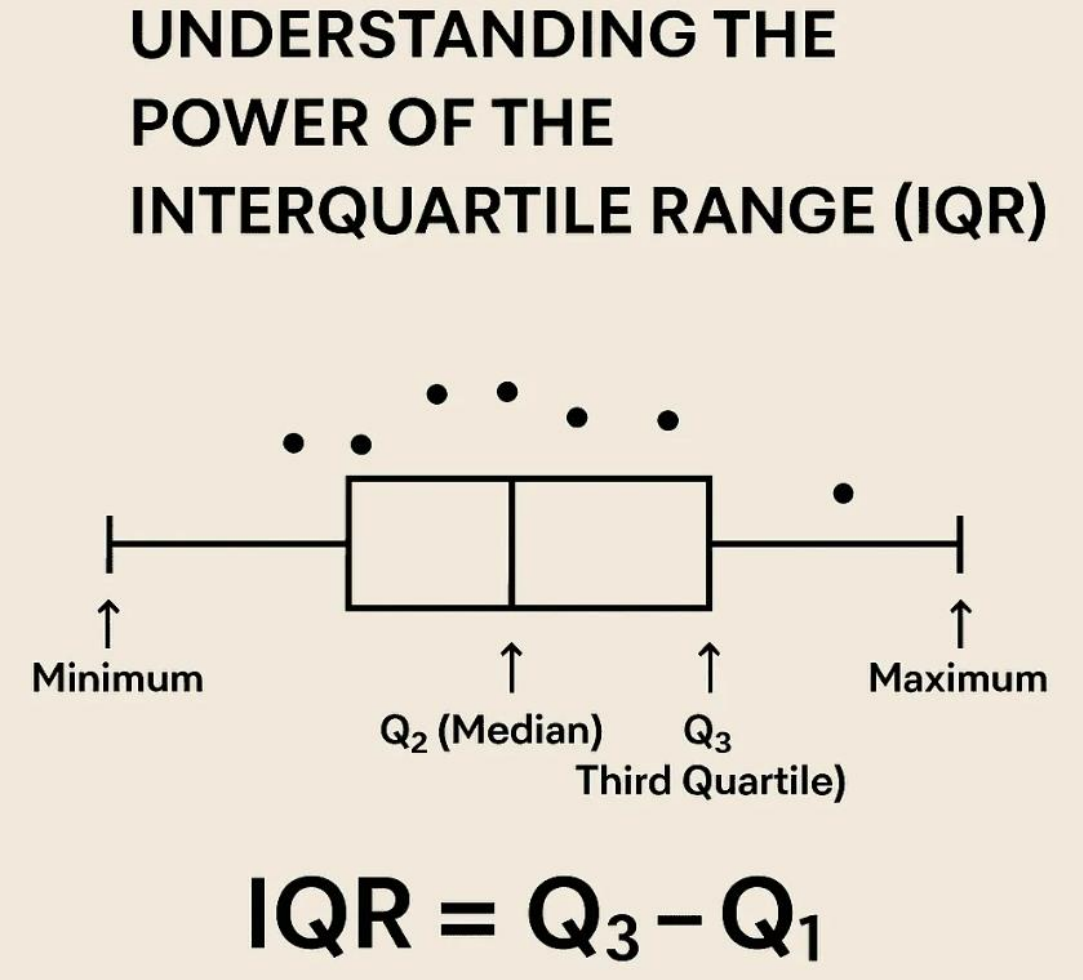
- **# Extended percentiles**
  - percentiles = df['price'].quantile([0.05, 0.25, 0.50, 0.75, 0.95])

Percentile	Meaning
5th	Worst 5% of days (tail risk)
25th (Q1)	Lower quartile
50th (Q2)	Median
75th (Q3)	Upper quartile
95th	Best 5% of days

Understanding Distribution Shape

Interquartile range:

- IQR = Q3 - Q1 # Spread of middle 50%
- Measures typical volatility
- Less affected by extreme outliers



Skewness (Asymmetry)

- skew = df['return'].skew()

Value	Interpretation
> 0	Right-skewed (more extreme positive returns)
< 0	Left-skewed (more extreme negative returns)
≈ 0	Symmetric distribution

Kurtosis (Tail Risk)

- kurt = df['return'].kurtosis()

Value	Interpretation
> 0	Fat tails (more extreme events than normal)
< 0	Thin tails (fewer extremes)
= 0	Normal distribution

Categorical Variables Analysis

- **Frequency Tables:**
  - # Count occurrences
  - df['sector'].value_counts()

  - # Sorted by frequency
  - df['sector'].value_counts().sort_values(ascending=False)

  - # As percentages
  - df['sector'].value_counts(normalize=True) * 100

Categorical Variables Analysis

- **Mode (Most Common):**
  - mode = df['sector'].mode()

- **Measuring Relationships Between Variables:**
  - # Correlation between two variables
  - corr = df['return'].corr(df['volume'])

  - # Correlation matrix (all pairs)
  - corr_matrix = df[['return', 'volume', 'volatility']].corr()

Correlation Scale

Value	Relationship
+1	Perfect positive
0.7 to 1	Strong positive
0.3 to 0.7	Moderate positive
-0.3 to 0.3	Weak/None
-0.7 to -0.3	Moderate negative
-1 to -0.7	Strong negative
-1	Perfect negative

Covariance vs Correlation

- **Covariance:**
  - cov = df['return'].cov(df['market_return’])
- Measures how variables move together
- Not standardized - hard to interpret magnitude
- Depends on units of measurement

- **Correlation:**
  - corr = df['return'].corr(df['market_return’])
- Standardized covariance (-1 to +1)
- Easy to interpret
- Unit-independent

Best Practices

- Always check df.isnull().sum() first
- Verify data types with df.dtypes
- Use df.describe() for quick overview
- Document your steps in Markdown cells

Complete Workflow Example

```
import pandas as pd
import numpy as np

# 1. Load data
df = pd.read_csv('stock_returns.csv')

# 2. Inspect
print(df.head())
print(df.isnull().sum())

# 3. Identify variable types
numerical = ['daily_return', 'volume']
categorical = ['sector']

# 4. Descriptive statistics
print(df[numerical].describe())

# 5. Risk metrics
volatility = df['daily_return'].std() *
np.sqrt(252)
skewness = df['daily_return'].skew()

# 6. Relationships
correlation =
df['daily_return'].corr(df['market_return'])

# 7. Report findings
print(f"Annual Volatility: {volatility:.2%}")
print(f"Skewness: {skewness:.3f}")
print(f"Market Correlation: {correlation:.3f}")
```

Summary

- Jupyter Notebook - Interactive analysis environment
- CSV Structure - Rows = observations, Columns = variables
- Data Inspection - Always check shape, types, missing values
- Numerical Statistics - Mean, median, std, quartiles, skew, kurtosis
- Categorical Statistics - Frequency tables, mode
- Relationships - Covariance and correlation

Best Practices

- Always inspect data before analysis
- Check for missing values
- Annualize volatility for comparability
- Use median for skewed distributions
- Correlation ≠ Causation